

TEXTBOOK EDITION • HIGH-LEVEL ARCHITECTURE & FINTECH RAILS

Scalable Financial System Design & UPI 2.0

50k TPS Payment Switch, Distributed Sagas, Idempotency & Core Banking Rails

TARGET ROLE	Developer (3+ Years Experience)
DEPARTMENT	Strategic Projects & New Age Tech, Bengaluru
PREPARED FOR	Bipin Yadav (SDE II, 4+ YoE)
ESTIMATED STUDY TIME	20 Hours
DIFFICULTY LEVEL	Architectural Mastery
TEACHING FORMAT	Hinglish Mental Models + Production FinTech

Table of Contents

Chapter 01 — Financial System Design Fundamentals & The 10–Step Interview Framework	Study Time: 35m
Chapter 02 — System Design: High–Throughput Payment Switch & Settlement Engine	Study Time: 35m
Chapter 03 — Indian Banking Rails, UPI 2.0 Architecture & Digital Payments	Study Time: 35m
Chapter 04 — Core Banking Transaction Processing, Double–Entry Ledgers & Sagas	Study Time: 35m
Chapter 05 — System Design Last–Minute Interview Cheatsheet	Study Time: 35m

CHAPTER 01

Financial System Design Fundamentals & The 10–Step Interview Framework

PRIORITY

● High Priority

DIFFICULTY

Architectural Mastery

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

Financial System Design Fundamentals & The 10–Step Interview Framework

1. Why This Matters

For a **Developer with 3+ years experience** applying to IDFC FIRST Bank's Strategic Projects, the System Design round is often the primary filter separating junior coders from true backend engineers. In this round, the interviewer provides an intentionally vague prompt (e.g., *"Design a UPI Payment Switch"* or *"Design a High-Throughput Transaction Ledger"*). They evaluate whether you panic, jump immediately to drawing boxes, or systematically guide the interview through requirements, scale calculations, data consistency, idempotency, and failure mitigation.

2. Prerequisites

- Client-server architecture, HTTP/HTTPS, REST, and gRPC.
- Relational databases (PostgreSQL/MySQL), NoSQL, Caching (Redis), and Message Brokers (Kafka/RabbitMQ).
- Basic networking: DNS, Load Balancers (L4 vs L7), Reverse Proxies.

3. Concept: The 10–Step Banking System Design Framework

```
flowchart TD
    S1["1. Clarify Requirements (Functional & Non-Functional)"] --> S2["2. Scale Estimation (TPS, Storage, Bandwidth)"]
    S2 --> S3["3. API Design (Idempotent Contracts, REST/gRPC)"]
    S3 --> S4["4. Data Model & Storage Choice (Relational, ACID, Polyglot)"]
    S4 --> S5["5. High-Level Architecture (Gateway, Microservices, DB)"]
    S5 --> S6["6. Core Component Deep Dive (Ledger, Queues, Caching)"]
    S6 --> S7["7. Concurrency & Consistency (Pessimistic Locks, Sagas)"]
    S7 --> S8["8. Resiliency & Failure Modes (Circuit Breakers, DLQs)"]
    S8 --> S9["9. Security & Compliance (PCI-DSS, RBI Localization, HSM)"]
    S9 --> S10["10. Observability & Trade-offs Defense"]
```

Step 1: Clarify Requirements

Never start drawing architecture without locking down scope: – **Functional Requirements:** What must the system do? (e.g., User initiates transfer, funds are debited from A and credited to B, audit trail recorded, notifications dispatched). – **Non-Functional Requirements:** – **Availability vs Consistency (CAP):** Banking systems are **CP systems** (Consistency & Partition Tolerance over Availability). Money cannot be double-spent. – **Latency SLA:** < 500ms for p99 API response. – **Durability:** Zero data loss; 100% auditability.

Step 2: Scale & Capacity Estimation

- **Traffic:** Peak Transactions Per Second (TPS). Example: 10,000 TPS peak during Diwali sales.
- **Storage:** 10,000 txns/sec * 86,400 sec/day ≈ 864 million txns/day. At 500 bytes per ledger record: ≈ 432 GB/day ⇒ ≈ 150 TB/year. *Architectural Conclusion:* Single database node will fail; requires horizontal sharding by `account_id` and cold-data archival to object storage (Cloud Storage / S3).

4. Simple Example: Back-of-the-Envelope Calculation Template

Scale Template for IDFC Systems:

- Daily Active Users (DAU): 10 Million
- Average transactions per user per day: 2
- Total daily transactions: 20 Million
- Average TPS = 20M / 86,400 ≈ 230 TPS
- Peak TPS (5x multiplier for festival bursts): 230 * 5 ≈ 1,150 TPS
- Storage per record: ~1 KB (PII, tokens, signatures, audit data)
- Daily storage: 20M * 1 KB = 20 GB/day
- 5-Year Regulatory Archive: 20 GB * 365 * 5 ≈ 36.5 TB

5. Real-World Banking Example: The High-Level Architecture

flowchart TD

```
Client["Client (Mobile App / Netbanking)"] → CDN["Cloud Armor / WAF (DDoS Shield)"]
CDN → LB["Layer 7 Load Balancer (ALB / GCP HTTPS LB)"]
LB → GW["API Gateway / BFF (Auth, Rate Limiting, TLS Termination)"]
```

```
subgraph Microservices["Kubernetes Cluster (Private VPC)"]
  GW → AuthSvc["Auth & Token Service"]
  GW → TransferSvc["Transfer Orchestration Service"]
  TransferSvc → AccSvc["Account & Ledger Service"]
  TransferSvc → SwitchSvc["Payment Switch Adapter (NPCI/Visa)"]
end
```

```
subgraph DataTier["Data & Event Tier"]
  AccSvc → RedisCluster["Redis Cluster (Distributed Locks & Idempotency)"]
  AccSvc → PrimaryDB["PostgreSQL Primary (Write Master - Multi-AZ)"]
  PrimaryDB -.→ ReadReplica["PostgreSQL Read Replicas"]
  TransferSvc → Kafka["Apache Kafka (Ordered Financial Event Stream)"]
  Kafka → AuditSvc["Audit & Fraud Detection Service"]
  Kafka → NotifySvc["Transactional Notification Service"]
end
```

6. Code: Distributed Token Bucket Rate Limiter with Redis Lua Script

```
-- token_bucket.lua
-- Atomically checks and decrements available tokens in Redis
local key = KEYS[1]
local capacity = tonumber(ARGV[1])
local refill_rate = tonumber(ARGV[2]) -- tokens per second
local now = tonumber(ARGV[3])
local requested = tonumber(ARGV[4])

-- Get bucket state: [tokens, last_refreshed_timestamp]
local data = redis.call("HMGET", key, "tokens", "last_updated")
local tokens = tonumber(data[1])
local last_updated = tonumber(data[2])

if not tokens then
    tokens = capacity
    last_updated = now
else
    local delta = math.max(0, now - last_updated)
    tokens = math.min(capacity, tokens + delta * refill_rate)
    last_updated = now
end

if tokens >= requested then
    tokens = tokens - requested
    redis.call("HMSET", key, "tokens", tokens, "last_updated", last_updated)
    redis.call("EXPIRE", key, 3600)
    return 1 -- Allowed
else
    return 0 -- Rejected (HTTP 429 Too Many Requests)
end
```

7. How It Works Internally: CAP & PACELC Theorems in Banking

- **CAP Theorem:** In the presence of a network partition (P), a distributed system must choose between Consistency (C) and Availability (A). Banking systems choose **Consistency**: if the primary database partition cannot communicate with the secondary node, writes must reject rather than accept divergent balances.
- **PACELC Theorem:** Extends CAP: If there is a Partition (P), trade off Availability (A) and Consistency (C); **Else (E)**, trade off Latency (L) and Consistency (C). In normal operations, banking switches accept higher write latency (synchronous replica replication) to guarantee strict serializable consistency.

8. Common Mistakes

1. **Dumping Technology Names Without Justification:** Saying "I will use Kafka and Cassandra because they are scalable" fails an interview. State why: "I select Kafka for the event stream because it provides partitioned, strictly ordered event logs per `account_id` and consumer group replayability."
2. **Ignoring Single Points of Failure (SPOF):** Forgetting to declare database read replicas, multi-AZ active-passive failover, or multi-region disaster recovery (RPO and RTO).

3. **Omitting Financial Idempotency & Reconciliation:** In a banking system design, if you do not mention idempotency keys and end-of-day reconciliation with NPCI/Visa, your design is incomplete.

9. Performance / Complexity Matrix

Component	Technology Choice	Latency SLA	Availability Target
API Gateway	Kong / Envoy / Express Gateway	< 10ms	99.99%
Distributed Cache	Redis Cluster (In-Memory)	< 2ms	99.95%
Event Stream	Apache Kafka	< 15ms	99.99%
Core Relational DB	PostgreSQL Multi-AZ with WAL	< 25ms	99.999% (Five Nines)
Audit Log Store	OpenSearch / GCP Cloud Storage	Async	99.999999999% (11 Nines)

10. Interview Questions (Easy → Medium → Hard)

Easy

- **Q:** What is the difference between horizontal scaling and vertical scaling?

Medium

- **Q:** Explain how you choose between Redis and Memcached for caching banking session states.

Hard

- **Q:** Design a distributed ledger that handles 50,000 transactions per second across 100 million accounts while preventing hot-spotting on high-volume merchant accounts (e.g., Amazon or Swiggy receiving 5,000 credits/sec).

11. Follow-up Questions from Interviewer

- "If a merchant account receives 5,000 transfers per second, row-level locking on that single merchant `account_id` will serialize and bottleneck the entire database. How do you solve hot-spot account write contention?" (Answer: Sharded balance accounts / Bank Sub-accounts: Split the merchant's balance into 50 sub-accounts in the database. Inbound credits randomly distribute across sub-accounts without locking contention; total balance is the sum of sub-accounts).
- "What is the difference between RPO (Recovery Point Objective) and RTO (Recovery Time Objective) in disaster recovery planning?"

12. Model Answer: PACELC & Synchronous DB Replication in FinTech

BANKING & FINTECH ARCHITECTURE SCENARIO

Interviewer: "Why don't we use asynchronous database replication for the primary financial ledger?"

Model Answer: "In high-throughput web systems, asynchronous database replication is standard because it maximizes write throughput by allowing the primary node to acknowledge commits before replicas receive the WAL stream.

However, in a banking ledger governed by the **PACELC theorem (PC/EC)**, asynchronous replication introduces the risk of data loss: – If the primary database crashes before WAL records are replicated to the standby replica, the promoted replica suffers from **unreplicated transactions**. – In a banking system, that loss represents money that was debited from a customer or credited to a merchant that has permanently vanished from the failover node (a catastrophic non-zero **Recovery Point Objective (RPO)**).

Therefore, core banking ledgers mandate **Semi-Synchronous or Synchronous Multi-AZ Replication** (such as PostgreSQL with `synchronous_commit = on` and at least one synchronous standby). The primary blocks until at least one independent replica has flushed the WAL to disk, guaranteeing zero data loss (RPO = 0) in the event of an automated primary node failover."

13. Practical Exercise

Review the 10-step template and practice driving a 5-minute verbal walkthrough of designing a **High-Priority OTP Notification Engine** (SMS, WhatsApp, Push) that must deliver OTPs within 3 seconds.

14. Quick Revision

- Follow the 10-step framework: Requirements → Scale → API → Data → Architecture → Deep Dive → Consistency → Failure → Security → Observability.
- Banking systems are CP systems (Consistency > Availability).
- Shard databases by `account_id` or `customer_id` to distribute load evenly.
- Mitigate hot merchant accounts using distributed sub-account balance sharding.
- Synchronous WAL replication guarantees zero financial data loss (RPO = 0).

15. Interview Checklist

- ☐ Memorizes the 10-step system design sequence.
- ☐ Confidently calculates Peak TPS, Storage, and Memory requirements.
- ☐ Explains CAP and PACELC in the context of banking consistency.
- ☐ Solves the merchant hot-spot account contention bottleneck.



CHAPTER 01 SUMMARY & REVISION CHECKLIST



WHAT TO REMEMBER

- Client-server architecture, HTTP/HTTPS, REST, and gRPC.
- Relational databases (PostgreSQL/MySQL), NoSQL, Caching (Redis), and Message Brokers (Kafka/RabbitMQ).
- Basic networking: DNS, Load Balancers (L4 vs L7), Reverse Proxies.



CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 02

System Design: High-Throughput Payment Switch & Settlement Engine

PRIORITY

 High Priority

DIFFICULTY

Architectural Mastery

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

System Design: High-Throughput Payment Switch & Settlement Engine

1. Why This Matters

Designing a Payment System is the quintessential System Design interview question at IDFC FIRST Bank. The Strategic Projects team builds payment gateways, card switches, and UPI acquiring engines. An interviewer will push you deep into every architectural layer: from HTTP idempotency keys at the API gateway down to double-entry ledger database tables, Kafka dead-letter queues, and end-of-day bank reconciliation.

2. Prerequisites

- [06-SYSTEM-DESIGN/system-design-fundamentals.md](#)
- [11-BANKING-FINTECH/transaction-processing.md](#)
- Relational ACID transactions and distributed Sagas.

3. Concept: The Payment System Architecture

flowchart TD

```
MerchantApp["Merchant App / Client"] --> WAF["Cloud Armor / WAF (DDoS Shield)"]
WAF --> GW["API Gateway (Rate Limiter + Idempotency Filter)"]
```

```
subgraph PaymentPlatform["IDFC FIRST Bank Payment Platform"]
  GW --> PaySvc["Payment Orchestration Service"]
  PaySvc --> RiskSvc["Risk & Fraud Engine (Velocity Checks)"]
  PaySvc --> LedgerSvc["Double-Entry Ledger Service"]
  PaySvc --> RoutingSvc["Smart Payment Router"]
end
```

end

```
subgraph ExternalRails["External Payment Rails"]
  RoutingSvc --> NPCI["NPCI UPI Switch"]
  RoutingSvc --> VisaMC["Visa / Mastercard Gateway"]
  RoutingSvc --> CBS["Core Banking Solution (IDFC CBS)"]
end
```

end

```
subgraph AsyncTier["Asynchronous Processing Tier"]
  PaySvc --> Kafka["Apache Kafka (Payment Events Topic)"]
  Kafka --> ReconEngine["Reconciliation & Settlement Engine"]
  Kafka --> WebhookSvc["Merchant Webhook Notification Engine"]
  Kafka --> Analytics["Fraud & Regulatory Reporting"]
end
```

end

4. Requirements & Scale Estimation

Functional Requirements

1. **Execute Payment:** Merchant/User submits payment request with payment method (Card, UPI, Netbanking).
2. **Idempotency:** Guarantee that network retries or duplicate button clicks never cause duplicate debits.
3. **Ledger Integrity:** Maintain immutable double-entry records for all financial events.
4. **Reconciliation:** Match internal transaction state with daily settlement clearing files from NPCI/Visa.

Non-Functional Requirements

- **Consistency:** Strict consistency for balance deductions; zero double-spending.
- **Throughput:** Handle 5,000 peak TPS with < 1.5s p99 latency.
- **Availability:** 99.999% (Five Nines).
- **Security:** PCI-DSS Level 1 compliant; zero raw card data in application logs.

5. API Design & Data Schema

Core API Endpoint

```
POST /v1/payments/charges
Headers:
  Content-Type: application/json
  Authorization: Bearer <api_key>
  X-Idempotency-Key: <uuid_v4>

Request Body:
{
  "amount": 250000, // In smallest currency unit (2500.00 INR = 250000 paise)
  "currency": "INR",
  "customer_id": "cust_89123",
  "payment_method": {
    "type": "UPI",
    "vpa": "aarav@idfcbank"
  },
  "metadata": {
    "order_id": "ORD_991823"
  }
}
```

Relational Database Schema (PostgreSQL)

```
-- Payments Table (Order state machine)
CREATE TABLE payments (
  payment_id VARCHAR(36) PRIMARY KEY,
  idempotency_key VARCHAR(64) UNIQUE NOT NULL,
  customer_id VARCHAR(36) NOT NULL,
  amount BIGINT NOT NULL CHECK (amount > 0), -- Stored in paise
  currency VARCHAR(3) NOT NULL DEFAULT 'INR',
  status VARCHAR(20) NOT NULL CHECK (status IN ('CREATED', 'INITIATED', 'SUCCESS', 'FAILED',
'REFUNDED')),
  payment_channel VARCHAR(20) NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Ledger Entries (Double-Entry Bookkeeping)
CREATE TABLE ledger_entries (
  entry_id BIGSERIAL PRIMARY KEY,
  payment_id VARCHAR(36) NOT NULL,
  account_id VARCHAR(36) NOT NULL,
  direction VARCHAR(6) NOT NULL CHECK (direction IN ('DEBIT', 'CREDIT')),
  amount BIGINT NOT NULL CHECK (amount > 0),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (payment_id) REFERENCES payments(payment_id)
);

CREATE INDEX idx_payments_idempotency ON payments(idempotency_key);
CREATE INDEX idx_ledger_account ON ledger_entries(account_id, created_at);
```

6. Code: Payment State Machine & Idempotency Transition in Node.js

```
const PaymentStatus = {
  CREATED: 'CREATED',
  INITIATED: 'INITIATED',
  SUCCESS: 'SUCCESS',
  FAILED: 'FAILED'
};

class PaymentStateMachine {
  static isValidTransition(current, next) {
    const allowed = {
      [PaymentStatus.CREATED]: [PaymentStatus.INITIATED, PaymentStatus.FAILED],
      [PaymentStatus.INITIATED]: [PaymentStatus.SUCCESS, PaymentStatus.FAILED],
      [PaymentStatus.SUCCESS]: [], // Terminal state
      [PaymentStatus.FAILED]: [] // Terminal state
    };
    return allowed[current]?.includes(next) || false;
  }
}

async function updatePaymentStatus(paymentId, fromStatus, toStatus, dbClient) {
  if (!PaymentStateMachine.isValidTransition(fromStatus, toStatus)) {
    throw new Error(`Invalid state transition from ${fromStatus} to ${toStatus}`);
  }

  // Atomic conditional status update
  const result = await dbClient.query(
    `UPDATE payments
     SET status = $1, updated_at = NOW()
     WHERE payment_id = $2 AND status = $3`,
    [toStatus, paymentId, fromStatus]
  );

  if (result.rowCount === 0) {
    throw new Error('Concurrent state transition conflict. Aborting.');
```

7. How It Works Internally: The 3-Way Reconciliation Engine

```
flowchart TD
    subgraph ThreeWayMatch["Daily 3-Way Reconciliation Matching Engine"]
        Internal["1. Internal Ledger\n(Payment Orchestration DB)"]
        SwitchFile["2. Payment Gateway File\n(NPCI / Visa Raw Settlement File)"]
        BankCBS["3. Bank CBS Settlement\n(IDFC Core Banking Statement)"]

        Internal --> Engine["Daily Batch Matching Engine (Spark / Spring Batch)"]
        SwitchFile --> Engine
        BankCBS --> Engine

        Engine --> Matched["Matched & Settled ($100%)"]
        Engine --> Discrepancy["Discrepancy Queue (Investigate: One-Way Debit)"]
    end
```

Reconciliation Workflow:

1. Every night at 02:00 AM, the bank receives **Clearing and Settlement raw files** (e.g., NPCI `EOD_SETTLEMENT.txt`).

2. A distributed batch processor (Apache Spark or Spring Batch) parses the millions of records.
3. For every transaction reference, it checks:
4. Did the internal ledger record `SUCCESS` ?
5. Did NPCI confirm `SUCCESS` ?
6. Did the inter-bank nodal settlement credit the bank's clearing account?
7. Any mismatch triggers automated correction workflows (e.g., auto-reversals or credit adjustments) in accordance with RBI timelines.

8. Common Mistakes

1. **Storing Currency in Decimals/Floats:** Floats create IEEE rounding errors. Always store currency as **64-bit integer paise/cents** (`amount: 50000` = ₹500.00).
2. **Synchronous Webhooks:** Making synchronous HTTP calls to merchant webhooks inside the payment processing request blocks worker threads. Always push webhook events to **Kafka** and let dedicated asynchronous workers dispatch them with exponential backoff.
3. **Omitting Distributed Locking for Balances:** If two payments hit the same account concurrently, failing to lock or verify versions causes balance double-spending.

9. Performance / Complexity Matrix

Layer	Technology	Latency	Scalability Strategy
Idempotency Gate	Redis <code>SET NX</code>	< 2ms	Sharded Redis Cluster
Payment Router	Node.js Microservice	< 15ms	Horizontal Pod Autoscaling (HPA)
Transaction DB	PostgreSQL Multi-AZ	< 25ms	Partitioned by Date / Sharded by Customer ID
Asynchronous Bus	Apache Kafka	< 10ms	Partitioned by <code>customer_id</code> for in-order events

10. Interview Questions (Easy → Medium → Hard)

Easy

- **Q:** Why should monetary amounts be stored as integers (e.g., paise) rather than floating-point numbers in a payment database?

Medium

- **Q:** How do you guarantee that a merchant webhook notification is reliably delivered even if the merchant's server is down for 6 hours?

Hard

- **Q:** A payment switch experiences a network split while waiting for a response from NPCI. The client is waiting on an open HTTP connection. How should the switch handle the client response, the pending database record, and subsequent reconciliation?

11. Follow-up Questions from Interviewer

- "What is the Dead Letter Queue (DLQ) strategy for failed webhook delivery retries?" (Answer: Retry with exponential backoff: 5s, 30s, 5m, 30m, 2h. After max retries, move to DLQ and trigger email alert to merchant technical contact).
- "How do you partition a Kafka topic for payment events to ensure in-order processing of transactions for a single account?" (Answer: Use `account_id` as the Kafka message partition key. Kafka guarantees strict chronological order within a single partition).

12. Model Answer: Network Split During Payment Gateway Call

BANKING & FINTECH ARCHITECTURE SCENARIO

Interviewer: "If the payment service sends a debit request to the external banking switch, but the connection times out before receiving a response, what do you return to the user?"

Model Answer: "When an external payment switch call times out, the transaction is in an **IN-DOUBT / PENDING** state. We must NEVER assume failure and refund, nor assume success and fulfill the order.

1. **Immediate API Response:** We return HTTP 202 Accepted with status `PENDING` and the transaction reference to the client, displaying a 'Payment Processing' state.
2. **Active Status Polling:** A background worker immediately initiates **Status Inquiry APIs** to the external switch at 5s, 15s, and 30s intervals using the unique RRN.
3. **Terminal State Resolution:** If the switch confirms the debit succeeded, we transition the payment to `SUCCESS`, write double-entry ledger records, and dispatch a webhook. If confirmed failed, we transition to `FAILED`.
4. **Nightly Reconciliation Fallback:** If the status inquiry remains unresponsive, the transaction remains `PENDING` until the nightly settlement file resolves it with 100% finality."

13. Practical Exercise

Draw the high-level architecture diagram of a Payment Gateway on a whiteboard or paper, ensuring you include: API Gateway, Redis Idempotency Cache, Payment Orchestrator, PostgreSQL Ledger DB, Kafka Event Stream, and the Reconciliation Batch Worker.

14. Quick Revision

- Currency amounts must always be stored in smallest units (integer paise).

- Idempotency middleware intercepts requests via `X-Idempotency-Key` and Redis `SET NX`.
- Ledger entries are immutable: every transfer creates balanced Debits and Credits.
- Sagas coordinate distributed transaction steps with compensating reversals.
- End-of-day reconciliation guarantees that internal ledgers match bank switch clearing files.

15. Interview Checklist

- [] Draws the complete payment flow architecture confidently.
- [] Defines the relational schema with double-entry ledger entries.
- [] Explains the state machine transitions and prevents race conditions.
- [] Explains 3-way reconciliation against NPCI/Visa clearing files.

CHAPTER 02 SUMMARY & REVISION CHECKLIST

WHAT TO REMEMBER

- [06-SYSTEM-DESIGN/system-design-fundamentals.md]
(file:///home/bipin/Desktop/BankInterview/06-SYSTEM-DESIGN/system-design)
- [11-BANKING-FINTECH/transaction-processing.md]
(file:///home/bipin/Desktop/BankInterview/11-BANKING-FINTECH/transaction-p)
- Relational ACID transactions and distributed Sagas.

CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 03

Indian Banking Rails, UPI 2.0 Architecture & Digital Payments

PRIORITY

 High Priority

DIFFICULTY

Architectural Mastery

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

Indian Banking Rails, UPI 2.0 Architecture & Digital Payments

1. Why This Matters

You are interviewing for the **Strategic Projects** division within **IDFC FIRST Bank**, Bengaluru. IDFC FIRST Bank prides itself on cutting-edge digital banking infrastructure, instant account opening, and high-volume UPI/IMPS transaction switches. If you walk into this interview and only know generic CRUD web development, you will struggle when they ask about Indian payment rails. Demonstrating command over **UPI 2.0**, NPCI settlement switches, VPA routing, and idempotency guarantees will immediately set you apart as a top 5% candidate.

2. Prerequisites

- Client-server architecture, REST APIs, and asynchronous message queues.
- Basic understanding of public-key cryptography (digital signatures, PKI).

3. Concept

Indian Payment Rails Comparison Matrix

Rail	Governing Body	Settlement Type	Operating Hours	Max Limit / Txn	Typical Latency
UPI	NPCI	Real-Time Gross (Per-Txn)	24x7x365	₹1 Lakh (₹5 Lakh for hospitals/education)	1–3 seconds
IMPS	NPCI	Real-Time Gross	24x7x365	₹5 Lakh	2–5 seconds
NEFT	RBI	Deferred Net Settlement (DNS) in half-hourly batches	24x7x365	No minimum / No limit	30–60 mins
RTGS	RBI	Real-Time Gross Settlement (Continuous)	24x7x365	Min ₹2 Lakh / No upper limit	Instant (< 30s)

4. UPI 2.0 System Architecture & Key Entities

sequenceDiagram
 autonumber
 actor Payer as Payer (Customer)
 participant PayerApp as Payer PSP (e.g. PhonePe / IDFC App)
 participant NPCI as NPCI UPI Central Switch
 participant RemitterBank as Remitter Bank (IDFC FIRST CBS)
 participant BeneficiaryBank as Beneficiary Bank (e.g. HDFC Bank)
 actor Payee as Payee (Merchant / Friend)

 Payer->>PayerApp: 1. Initiate Transfer to payee@okaxis (₹2,000)
 PayerApp->>NPCI: 2. ReqVPA (Resolve VPA to Account Details)
 NPCI->>BeneficiaryBank: 3. Verify Account status & Name
 BeneficiaryBank-->>NPCI: 4. Payee details verified (John Doe)
 NPCI-->>PayerApp: 5. Display Payee Name to User

 Payer->>PayerApp: 6. Enters 6-digit mPIN
 Note over PayerApp: mPIN is encrypted via Common Library (CL) using NPCI Public Key
 PayerApp->>NPCI: 7. ReqPay (Encrypted mPIN payload + Idempotency Ref)
 NPCI->>RemitterBank: 8. ReqDebit (Debit Payer Account ₹2,000)
 Note over RemitterBank: Core Banking Solution (CBS) debits ₹2,000 atomically
 RemitterBank-->>NPCI: 9. ResDebit (SUCCESS, AuthCode, RRN)

 NPCI->>BeneficiaryBank: 10. ReqCredit (Credit Payee Account ₹2,000)
 Note over BeneficiaryBank: CBS credits ₹2,000 to John Doe
 BeneficiaryBank-->>NPCI: 11. ResCredit (SUCCESS)

 NPCI-->>PayerApp: 12. Transaction SUCCESS (RRN: 608129381023)
 PayerApp-->>Payer: 13. Displays Green Checkmark screen

The Key Participants:

- 1. Payer (Remitter) & Payee (Beneficiary): End customers or merchants.

2. **Payment Service Provider (PSP) App:** The frontend application (IDFC FIRST Mobile App, Google Pay, CRED).
 3. **Payer / Payee PSP (Bank):** The acquiring or routing bank providing the API gateway for the PSP app.
 4. **NPCI (National Payments Corporation of India):** The central switch coordinating routing, authentication payloads, and inter-bank multilateral netting settlements.
 5. **Remitter Bank (Issuer):** The bank holding the payer's account (e.g., IDFC FIRST Bank). Validates the encrypted mPIN and debits the core balance.
 6. **Beneficiary Bank (Acquirer):** The bank holding the payee's account. Receives credit instruction from NPCI and credits the recipient.
-

5. Real-World Banking Example: The Dreaded "Debit Success, Credit Timeout"

What happens when: - **Remitter Bank (IDFC FIRST):** Successfully debits ₹5,000 from customer's account. - **NPCI → Beneficiary Bank:** Network cable cut or Beneficiary Bank CBS hangs. - **Result:** Payer's money is debited, but Payee does not receive credit.

The Resolution Protocol (RBI T+1 Auto-Reversal Mandate):

1. **Deemed Success vs Deemed Failure:** If the credit request times out, NPCI initiates automated **Transaction Status Inquiries (ReqHbt / UPI Check Status API)** every few minutes.
 2. **Auto-Reversal (Credit Reversal):** If Beneficiary Bank confirms no credit took place, NPCI transmits a **ReqReversal** to Remitter Bank to refund the ₹5,000 back to the customer.
 3. **Turnaround Time (TAT) Rules:** RBI mandates that failed UPI transactions where account is debited must be reversed within **T+1 business days**. Failure to do so incurs an automatic statutory penalty of **₹100 per day** paid directly to the customer.
-

6. Code: Generating and Enforcing UPI Idempotency Keys in Node.js

```
const crypto = require('crypto');

/**
 * Generates an immutable, collision-resistant Retrieval Reference Number (RRN)
 * compliant with NPCI 12-digit format:
 * Format: YDDDDHHNNNNNN (Year last digit, Day of Year, Hour, 6-digit sequence)
 */
function generateUPIReferenceNumber(counter = 1) {
  const now = new Date();
  const yearLastDigit = now.getFullYear() % 10;

  // Day of year (001 - 366)
  const startOfYear = new Date(now.getFullYear(), 0, 0);
  const diff = now - startOfYear;
  const oneDay = 1000 * 60 * 60 * 24;
  const dayOfYear = String(Math.floor(diff / oneDay)).padStart(3, '0');

  const hour = String(now.getHours()).padStart(2, '0');
  const sequence = String(counter % 1000000).padStart(6, '0');

  return `${yearLastDigit}${dayOfYear}${hour}${sequence}`;
}

console.log('Sample NPCI RRN:', generateUPIReferenceNumber(45291));
```

7. How It Works Internally: The Common Library (CL) & mPIN Security

How does the NPCI prevent malicious PSP apps (like a third-party app) from stealing a user's 6-digit ATM/mPIN?

```
flowchart LR
    subgraph MobileDevice ["User Mobile Device (Sandbox)"]
        UI["App UI"]
        CL["NPCI Common Library (CL SDK)\nIsolated Native Binary"]
    end
    subgraph Backend ["Banking Switch"]
        HSM["Hardware Security Module (HSM)\nStores Banking Master Keys"]
    end
    UI -->|Requests PIN Entry| CL
    CL -->|Collects PIN securely| CL
    CL -->|Encrypts with NPCI Public Key| Payload["Encrypted PIN Block (PKI)"]
    Payload --> HSM
```

- Every UPI app must embed the **NPCI Common Library (CL SDK)** as an isolated native binary.
- When the user types their mPIN, the PSP app's JavaScript layer has **zero access to the keyboard events**.
- The CL encrypts the raw PIN using **RSA-2048 / 3DES** encryption inside a hardware security enclave using NPCI's public key.
- Only the **Hardware Security Module (HSM)** inside the Remitter Bank's core data center holds the corresponding private key to decrypt and verify the mPIN!

8. Common Mistakes

1. **Confusing VPA with Account Numbers:** Virtual Payment Addresses (`username@idfcbank`) decouple sensitive bank account numbers and IFSC codes from merchants. The mapping is resolved dynamically via NPCI central directory.
2. **Treating Payment Statuses as Binary (Success/Fail):** Payment gateways have an essential third state: **PENDING / IN-DOUBT**. Never trigger a refund or repeat debit on an "IN-DOUBT" status until the reconciliation loop or check-status API returns a terminal state.
3. **Assuming UPI Settlements are Real-Time Between Banks:** While customer accounts are debited and credited instantly, the actual **multilateral inter-bank settlement** between IDFC FIRST Bank and other banks occurs in discrete net clearing cycles managed by RBI and NPCI multiple times daily.

9. Performance / Complexity Matrix

Metric	UPI	IMPS	NEFT	RTGS
Switch Latency (SLA)	< 2 seconds	< 5 seconds	30–60 minutes	< 30 seconds
System Throughput	> 15,000 TPS nationwide	≈ 2,000 TPS	Batch files	Continuous gross
Idempotency Lifetime	48 hours	24 hours	N/A (Batch ref)	N/A

10. Interview Questions (Easy → Medium → Hard)

Easy

- **Q:** What is a VPA in UPI and what role does NPCI play during a UPI transaction?

Medium

- **Q:** What is the difference between IMPS and UPI? If IMPS already existed, why was UPI created?

Hard

- **Q:** Design an end-to-end reconciliation engine for IDFC FIRST Bank that processes 20 million daily UPI settlement records against NPCI raw clearing files. How do you detect and resolve one-way debits and settlement mismatches?

11. Follow-up Questions from Interviewer

- "What is a Mandate in UPI 2.0 (Autopay) and how does pre-authorization work for IPO subscriptions or recurring subscriptions?"
- "If NPCI returns a **TIMEOUT** during a `ReqCredit` call, what exact state machine transition must the Remitter Bank's transaction record undergo?"

12. Model Answer: Why UPI Exists Over IMPS

BANKING & FINTECH ARCHITECTURE SCENARIO

Interviewer: "If IMPS already provided 24x7 instant interbank transfers, why did NPCI design UPI?"

Model Answer: "While IMPS established the underlying real-time clearing switch, it suffered from severe friction that prevented mass consumer and merchant adoption: 1. **Frictional Addressing:** IMPS required exchanging sensitive 11-digit account numbers and 11-character IFSC codes (or MMID), raising security concerns and high error rates during manual entry. UPI introduced the **Virtual Payment Address (VPA)**, abstracting account details behind an intuitive identifier (`name@idfc`). 2. **Pull Payments (Collect Requests):** IMPS was strictly a 'Push' architecture (payer sends money). UPI introduced bidirectional protocols: 'Push' (pay) and 'Pull' (merchants requesting funds / collect requests / autopay mandates). 3. **Interoperable Single-Click 2FA:** IMPS relied on divergent, clunky bank OTPs and SMS gateways. UPI standardized the **NPCI Common Library (CL)**, providing seamless 2-factor authentication (Device Fingerprint + 6-digit mPIN) across any interoperable PSP application. 4. **Standardized QR Code Rails:** UPI homogenized Bharat QR and dynamic payment strings, turning any smartphone into an instant point-of-sale terminal."

13. Practical Exercise

Review `05-SQL-DBMS/schema.sql` and note how the `transactions` table records `payment_channel` with explicit values (`'UPI'` , `'NEFT'` , `'RTGS'` , `'IMPS'`) and stores `transaction_reference` as an immutable lookup token.

14. Quick Revision

- UPI 2.0 = NPCI Central Switch + Remitter Bank (debit) + Beneficiary Bank (credit) + PSP Apps.
- Addressing is decoupled via Virtual Payment Address (VPA).
- mPIN is encrypted by the NPCI Common Library native binary using public key cryptography before touching the application layer.
- RBI T+1 rule mandates automated refunds for one-way debits with a ₹100/day delay penalty.
- Transactions are 3-state: `SUCCESS` , `FAILED` , and `IN-DOUBT` (requires reconciliation).

15. Interview Checklist

- [] Can draw the 13-step sequence diagram of a UPI transaction from memory.
- [] Explains how the Common Library protects mPIN from rogue mobile apps.
- [] Articulates the difference between Push and Pull transactions.
- [] Explains Deemed Success and T+1 auto-reversal mandates.



CHAPTER 03 SUMMARY & REVISION CHECKLIST



WHAT TO REMEMBER

- Client-server architecture, REST APIs, and asynchronous message queues.
- Basic understanding of public-key cryptography (digital signatures, PKI).
- Remitter Bank (IDFC FIRST): Successfully debits ₹5,000 from customer's account.



CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 04

Core Banking Transaction Processing, Double-Entry Ledgers & Sagas

PRIORITY

● High Priority

DIFFICULTY

Architectural Mastery

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

Core Banking Transaction Processing, Double-Entry Ledgers & Sagas

1. Why This Matters

In banking applications at IDFC FIRST Bank, money is never simply an integer updated in a single database column (`UPDATE accounts SET balance = balance + 100`). Real financial platforms use **immutable double-entry bookkeeping ledgers**, **distributed sagas**, and **strict idempotency filters**. In a 3+ YoE interview, if you propose mutating a single balance column without an immutable ledger audit trail, your architecture will be disqualified immediately.

2. Prerequisites

- Relational database transactions (ACID, isolation levels).
- Basic accounting principles (Debits and Credits).
- Distributed systems concepts (REST APIs, message brokers).

3. Concept

1. The Double-Entry Bookkeeping Principle

In accounting, money is neither created nor destroyed; it simply transfers between accounts. $\Sigma \text{Debits} = \Sigma \text{Credits}$ - **Debit**: An increase in Assets or Expenses, or a decrease in Liabilities or Equity. - **Credit**: An increase in Liabilities or Equity, or a decrease in Assets or Expenses. - **The Ledger Invariant**: Every financial transaction must insert **at least two immutable ledger entries**: one debit and one credit. The net sum of the entries for any transaction must strictly equal **zero**.

```

flowchart LR
    subgraph Transfer["Transfer: Aarav (Payer) pays Priya (Payee) ₹2,000"]
        direction TB
        E1["Ledger Entry 1:\nAccount: Aarav (Customer Liability)\nType: DEBIT (Balance Decreases) -₹2,000"]
        E2["Ledger Entry 2:\nAccount: Priya (Customer Liability)\nType: CREDIT (Balance Increases) +₹2,000"]
        Sum["Net Ledger Sum = ₹0.00 (Balanced!)"]
    end
    E1 --> Sum
    E2 --> Sum

```

4. Simple Example: Double-Entry Ledger Schema

```

CREATE TABLE ledger_entries (
    entry_id INTEGER PRIMARY KEY AUTOINCREMENT,
    transaction_reference VARCHAR(64) NOT NULL,
    account_id INTEGER NOT NULL,
    direction VARCHAR(6) NOT NULL CHECK (direction IN ('DEBIT', 'CREDIT')),
    amount DECIMAL(15, 2) NOT NULL CHECK (amount > 0),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (account_id) REFERENCES accounts(account_id)
);

-- Aarav transfers 2000 to Priya:
-- Entry 1 (Debit Aarav):
INSERT INTO ledger_entries (transaction_reference, account_id, direction, amount)
VALUES ('TXN-1001', 101, 'DEBIT', 2000.00);

-- Entry 2 (Credit Priya):
INSERT INTO ledger_entries (transaction_reference, account_id, direction, amount)
VALUES ('TXN-1001', 103, 'CREDIT', 2000.00);

```

5. Real-World Banking Example: Distributed Transaction Across Microservices (The Saga Pattern)

In a microservices architecture: - **Account Service:** Manages balance deductions (PostgreSQL). - **Payment Switch Service:** Talks to NPCI / Visa / Mastercard. - **Notification Service:** Dispatches transactional SMS / Email (Kafka).


```
sequenceDiagram
    autonumber
    participant Client as User Client
    participant Orchestrator as Transfer Saga Orchestrator
    participant AccService as Account Microservice
    participant Switch as Payment Gateway Switch
    participant Notify as Notification Service

    Client->>Orchestrator: 1. POST /transfers (Idempotency Key: IDEM-789)
    Orchestrator->>AccService: 2. Reserve / Hold Balance ₹5,000
    AccService-->>Orchestrator: 3. Balance Reserved (Hold ID: H-123)

    Orchestrator->>Switch: 4. Execute NPCI Wire Transfer
    alt Switch Transfer Fails (e.g. Beneficiary Bank Down)
        Switch-->>Orchestrator: 5. ERROR: Route Unavailable
        Note over Orchestrator: COMPENSATING TRANSACTION TRIGGERED
        Orchestrator->>AccService: 6. Release Hold / Refund ₹5,000 (Compensate)
        AccService-->>Orchestrator: 7. Balance Restored
        Orchestrator-->>Client: 8. Transfer Failed (No money lost)
    else Switch Transfer Succeeds
        Switch-->>Orchestrator: 5. SUCCESS: RRN 9028192
        Orchestrator->>AccService: 6. Commit Hold to Permanent Debit
        Orchestrator->>Notify: 7. Dispatch SMS Event
        Orchestrator-->>Client: 8. Transfer Successful!
    end
end
```

6. Code: Production-Grade Idempotency Engine in Express / Node.js

```

const redis = require('redis');
const redisClient = redis.createClient();

/**
 * Express Middleware: Guarantees exactly-once payment processing
 * using an Idempotency Key header.
 */
function idempotencyMiddleware(ttlSeconds = 86400) { // 24-hour retention
  return async (req, res, next) => {
    const idempotencyKey = req.headers['x-idempotency-key'];

    if (!idempotencyKey) {
      return res.status(400).json({ error: 'Missing required X-Idempotency-Key header'
});
    }

    const cacheKey = `idempotency:${idempotencyKey}`;

    // Step 1: Check if key exists or acquire lock (SET NX)
    // Returns 'OK' if key was successfully set, null if key already existed
    const acquired = await redisClient.set(cacheKey, JSON.stringify({ status: 'PROCESSING'
}), {
      NX: true,
      EX: ttlSeconds
    });

    if (!acquired) {
      // Key already exists! Fetch existing result or check progress
      const existingRecord = JSON.parse(await redisClient.get(cacheKey));

      if (existingRecord.status === 'PROCESSING') {
        return res.status(409).json({
          error: 'A request with this idempotency key is currently processing. Please
wait.'
        });
      }

      // Return cached previous response (replay response!)
      return res.status(existingRecord.statusCode).json(existingRecord.responseBody);
    }

    // Step 2: Intercept response to cache final outcome
    const originalSend = res.send.bind(res);
    res.send = (body) => {
      const parsedBody = JSON.parse(body);
      // Save final terminal outcome to Redis
      redisClient.set(cacheKey, JSON.stringify({
        status: 'COMPLETED',
        statusCode: res.statusCode,
        responseBody: parsedBody
      }), { EX: ttlSeconds });

      return originalSend(body);
    };

    next();
  };
}

```

7. How It Works Internally: Two-Phase Commit (2PC) vs Saga

Dimension	Two-Phase Commit (2PC)	Distributed Saga Pattern
Coordination	Central Transaction Coordinator	Saga Orchestrator or Choreography (Events)
Locking	Holds locks across all nodes during prepare phase	No cross-service distributed locks! Local DB commits immediately.
Failure Recovery	Automatic rollback via coordinator	Compensating Transactions undo previous local commits.
Latency & Scale	Poor scalability; slowest node delays all locks	High throughput & fault tolerant.
Consistency	Strict immediate consistency	Eventual consistency with intermediate states.



BANKING & FINTECH ARCHITECTURE SCENARIO

Why modern banking microservices use Sagas: In a modern cloud deployment, holding database locks across different microservices over network boundaries (2PC) creates single points of failure and catastrophic latency bottlenecks. Sagas execute independent local transactions and reverse failed steps using **compensating actions** (e.g., if a debit occurs but credit fails, issue a credit compensation).

8. Common Mistakes

- Mutating Balance Without Writing Ledger Records:** Calculating `balance = balance - 50` in-place leaves zero audit trail. If a customer questions a statement, the bank cannot prove where the money went. **Current balance should always be verifiable as the sum of all historical ledger entries.**
- Missing Distributed Idempotency Keys:** If a mobile client experiences a network glitch after sending a payment request, the user presses the "Pay" button a second time. Without an idempotency key, the server debits the account twice.
- Designing Non-Idempotent Compensations in Sagas:** Compensating transactions (refunds/cancellations) can *also* fail or be retried multiple times. Compensating transactions must themselves be strictly idempotent!

9. Performance / Complexity Matrix

Ledger Model	Read Complexity (Balance)	Write Complexity	Auditability
Naive Balance Update	O(1)	O(1)	Zero (Dangerous)
Pure Immutable Ledger (Sum of entries)	O(N) (Requires caching snapshot)	O(1) append-only	100% Perfect
Hybrid (Snapshot Balance + Ledger Table)	O(1) read snapshot	O(1) append + lock update	Excellent & Production Standard

10. Interview Questions (Easy → Medium → Hard)

Easy

- Q: What is the core rule of double-entry bookkeeping?

Medium

- Q: How do you implement idempotency for an HTTP `POST /api/v1/payments` endpoint? What happens if the second request arrives while the first is still processing?

Hard

- Q: Explain the difference between Orchestration-based Sagas and Choreography-based Sagas in financial systems. When would you strictly mandate an Orchestrator?

11. Follow-up Questions from Interviewer

- "In a double-entry system, how do you handle balance caching without desynchronizing from the immutable ledger table?"
- "What is 3-Way Reconciliation between the Bank's Core Ledger, the Payment Gateway Switch, and the Settlement Bank (NPCI/RBI)?"

12. Model Answer: Orchestration vs Choreography in Banking Sagas

BANKING & FINTECH ARCHITECTURE SCENARIO

Interviewer: "Why do core banking architectures typically prefer an Orchestration Saga over Choreography for fund transfers?"

Model Answer: "While Choreography (where services react autonomously to domain events on Kafka) works well for loosely coupled workflows like order fulfillment, banking fund transfers almost universally demand **Orchestration**.

1. **Centralized State Machine & Auditability:** A transfer involves strict legal states (**INITIATED** , **RESERVED** , **SWITCH_IN_FLIGHT** , **SETTLED** , **COMPENSATED**). An Orchestrator maintains a centralized state machine in a durable database, allowing operators to inspect the exact status of every flight transaction in real time.
2. **Deterministic Error & Compensation Handling:** If a transfer fails at Step 3, the orchestrator explicitly invokes compensating transactions in reverse order with exponential backoff. In choreography, coordinating multi-step compensations across asynchronous event topics introduces cyclic dependencies and high debugging complexity.
3. **Regulatory Compliance & Timeouts:** RBI regulations impose strict SLAs on transaction resolution. An orchestrator can easily manage timeouts and circuit breakers, escalating to manual operations queues if compensation retries are exhausted."

13. Practical Exercise

Review the **transactions** and **payments** tables in **05-SQL-DBMS/schema.sql** . Notice that: 1. **payments** has a **UNIQUE** constraint on **idempotency_key VARCHAR(64) UNIQUE NOT NULL** . 2. Any attempt to insert a duplicate payment with the same key triggers an immediate constraint violation at the database storage layer, preventing duplicate processing.

14. Quick Revision

- Double-entry: $\Sigma \text{Debits} = \Sigma \text{Credits}$; net sum of transaction is zero.
- Never mutate balances without creating append-only ledger entries.
- Idempotency middleware uses Redis **SET NX** with a 24-hour TTL and caches the response.
- Sagas replace 2PC across microservices using local transactions + compensating actions.
- Orchestrators maintain centralized state machines for regulatory compliance and auditability.

15. Interview Checklist

- [] Understands the accounting definition of debits and credits in bank liabilities.
- [] Can design a Redis-based idempotency lock engine with replay capability.
- [] Explains why Sagas are preferred over Two-Phase Commit in microservices.

- [] Articulates the difference between Orchestration and Choreography Sagas.



CHAPTER 04 SUMMARY & REVISION CHECKLIST



WHAT TO REMEMBER

- Relational database transactions (ACID, isolation levels).
- Basic accounting principles (Debits and Credits).
- Distributed systems concepts (REST APIs, message brokers).



CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases

CHAPTER 05

System Design Last-Minute Interview Cheatsheet

PRIORITY

● High Priority

DIFFICULTY

Architectural Mastery

EST. TIME

35–45 min

FREQUENCY

Very High (85%+)

System Design Last-Minute Interview Cheatsheet

1234 Numbers Every Engineer Must Know (Latencies)

L1 Cache Reference:	~0.5 ns
Branch Mispredict:	~5 ns
L2 Cache Reference:	~7 ns
Mutex Lock/Unlock:	~25 ns
Main Memory (RAM) Reference:	~100 ns
Compress 1KB with Snappy:	~3 µs
Send 1KB over 1Gbps Network:	~10 µs
Read 1MB Sequentially from SSD:	~1 ms
Disk Seek (HDD):	~10 ms
Send Packet Across Datacenter:	~0.5 ms
Round Trip (Mumbai to US-East):	~150 ms

High Availability & Uptime SLAs

Availability Target	Annual Allowed Downtime	Daily Allowed Downtime	Typical Tier
99% ("Two Nines")	3.65 days	14.4 minutes	Internal testing tools
99.9% ("Three Nines")	8.76 hours	43.2 seconds	Standard SaaS apps
99.99% ("Four Nines")	52.6 minutes	4.32 seconds	High-priority API gateways
99.999% ("Five Nines")	5.26 minutes	0.43 seconds	Core Banking Solution & Payment Rails

Redis Caching Strategies & Defenses

1. **Cache-Aside (Lazy Loading):** App reads cache. On miss, reads DB, writes to cache, returns. (Most common).

2. **Write-Through:** App writes to cache; cache synchronously writes to DB before acknowledging.
3. **Write-Behind (Write-Back):** App writes to cache; cache asynchronously batches writes to DB. (High write throughput, risk of data loss on crash).
4. **Cache Penetration Defense:** Querying keys that don't exist in DB.
5. *Fix:* Cache `null` with short TTL, or use a **Bloom Filter** at the gateway.
6. **Cache Stampede / Breakdown:** Hot key with high QPS expires; thousands of requests hit DB simultaneously.
7. *Fix:* Mutual exclusion lock (Mutex in Redis) or probabilistic early expiration (XFetch algorithm).

Apache Kafka Quick Architecture

- **Topic:** Logical stream of messages.
- **Partition:** Ordered, append-only log of immutable messages. The unit of parallelism.
- **Partition Key:** Messages with the same key (e.g., `account_id`) **always go to the same partition**, guaranteeing chronological ordering per account.
- **Consumer Group:** Multiple workers reading a topic. Each partition is consumed by strictly **one** consumer within a group.
- **Replication Factor:** N copies across brokers (typical in banking: 3 replicas with `min.insync.replicas = 2` and `acks = all` for zero message loss).

CHAPTER 05 SUMMARY & REVISION CHECKLIST

WHAT TO REMEMBER

- *Fix:* Cache `null` with short TTL, or use a Bloom Filter at the gateway.
- *Fix:* Mutual exclusion lock (Mutex in Redis) or probabilistic early expiration (XFetch algorithm).
- Topic: Logical stream of messages.

CANDIDATE SELF-CHECK

- ☐ I understand the fundamental mechanism
- ☐ I can explain it in natural Hinglish
- ☐ I can write or debug the core code/query
- ☐ I can defend trade-offs and edge cases